Laboratory Activity 3: Online Bookstore Management System

ITSD 82 Web Software Tools

Topic: Routing, Controllers, Views, Blade Templating, and Database Operations

---

Learning Objectives

Upon completion of this laboratory activity, students will be able to:

1. **Routing & HTTP Methods**
   - Define routes using GET, POST, PUT, PATCH, and DELETE methods
   - Implement route parameters with constraints
   - Create named routes, route prefixes, and route grouping
2. **Controllers**
   - Generate controllers using Artisan commands
   - Bind routes to controller methods
   - Implement resource controllers for CRUD operations
3. **Views & Blade Templating**
   - Use Blade syntax and directives (@if, @foreach, @extends, @yield, @section, @include)
   - Pass data from controllers to views using compact() and with()
   - Create layout templates and reusable partial views
   - Implement Blade components with slots
4. **Database Operations**
   - Create database migrations for multiple tables
   - Define Eloquent models with relationships
   - Implement database seeders and factories
   - Perform CRUD operations using Eloquent ORM
5. **Authentication**
   - Set up Laravel Breeze for user authentication
   - Implement authorization in views using @auth and @guest directives

---

Scenario: PageTurner Online Bookstore

Background Story

You have been hired as a junior web developer at **PageTurner Inc.**, a growing online bookstore company. The company wants to modernize their operations by building a comprehensive web-based management system. Your task is to develop the core functionality of the **PageTurner Online Bookstore Management System**.

The system should allow:
- **Administrators** to manage books, categories, and view orders
- **Customers** to browse books, view details, place orders, and write reviews
- **General visitors** to browse the catalog without making purchases

System Requirements

The bookstore system must include the following modules:

Module	Description
User Management	Registration, login, and profile management (via Laravel Breeze)
Category Management	CRUD operations for book categories
Book Management	CRUD operations for books with category assignment
Order Management	Create and view customer orders
Review System	Allow customers to write reviews for purchased books

Database Schema Design

Entity Relationship Diagram (ERD)

```
┌──────────────┐        ┌──────────────┐        ┌──────────────────┐
│   USERS      │        │ CATEGORIES   │        │     BOOKS        │
├──────────────┤        ├──────────────┤        ├──────────────────┤
│ id (PK)      │        │ id (PK)      │        │ id (PK)          │
│ name         │        │ name         │        │ category_id (FK)│
│ email        │        │ description │        │ title            │
│ password     │        │ created_at   │        │ author           │
│ role         │        │ updated_at   │        │ isbn             │
│ created_at   │        └──────────────┘        │ price            │
│ updated_at   │               │                │ stock_quantity   │
└──────────────┘               │                │ description      │
      │                        │                │ cover_image      │
      │                        └────────────────│ created_at       │
      │                                         │ updated_at       │
      │                                         └──────────────────┘
      │                                                  │
      │                                                  │
      │          ┌──────────────┐                        │
      │          │   ORDERS     │                        │
      │          ├──────────────┤                        │
      └─────────>│ id (PK)      │                        │
                 │ user_id (FK)│                        │
                 │ total_amount│                        │
                 │ status       │                        │
                 │ created_at   │                        │
                 │ updated_at   │                        │
                 └──────────────┘                        │
                       │                                 │
                       │     ┌──────────────────┐        │
                       │     │  ORDER_ITEMS     │        │
                       │     ├──────────────────┤        │
                       └────>│ id (PK)          │<───────┘
                             │ order_id (FK)    │
                             │ book_id (FK)     │
                             │ quantity         │
                             │ unit_price       │
                             │ created_at       │
                             │ updated_at       │
                             └──────────────────┘


┌──────────────────┐
│    REVIEWS       │
├──────────────────┤
```

```
│ id (PK)          │
│ user_id (FK)     │
│ book_id (FK)     │
│ rating           │
│ comment          │
│ created_at       │
│ updated_at       │
└──────────────────┘
```

Table 1: Database Tables Specification

Table Name	Primary Key	Foreign Keys	Description
`users`	id	-	Stores user accounts (customers & admins)
`categories`	id	-	Book categories/genres
`books`	id	category_id → categories	Book inventory
`orders`	id	user_id → users	Customer orders
`order_items`	id	order_id → orders, book_id → books	Individual items in orders
`reviews`	id	user_id → users, book_id → books	Customer book reviews

---

Table 2: Eloquent Relationships

Model	Relationship Type	Related Model	Method Name	Description
User	hasMany	Order	orders()	A user can have many orders
User	hasMany	Review	reviews()	A user can write many reviews
Category	hasMany	Book	books()	A category contains many books
Book	belongsTo	Category	category()	A book belongs to one category

Model	Relationship Type	Related Model	Method Name	Description
Book	hasMany	Review	reviews()	A book can have many reviews
Book	hasMany	OrderItem	orderItems()	A book can be in many order items
Order	belongsTo	User	user()	An order belongs to one user
Order	hasMany	OrderItem	orderItems()	An order has many items
OrderItem	belongsTo	Order	order()	An order item belongs to one order
OrderItem	belongsTo	Book	book()	An order item references one book
Review	belongsTo	User	user()	A review is written by one user
Review	belongsTo	Book	book()	A review is for one book

Table 3: Routes Specification

HTTP Method	URI	Controller Method	Route Name	Description
GET	/	HomeController@index	home	Homepage with featured books
GET	/categories	CategoryController@index	categories.index	List all categories
GET	/categories/create	CategoryController@create	categories.create	Show create category form
POST	/categories	CategoryController@store	categories.store	Store new category
GET	/categories/{category}	CategoryController@show	categories.show	Show single category

HTTP Method	URI	Controller Method	Route Name	Description
GET	/categories/{category}/edit	CategoryController@edit	categories.edit	Show edit category form
PUT	/categories/{category}	CategoryController@update	categories.update	Update category
DELETE	/categories/{category}	CategoryController@destroy	categories.destroy	Delete category
GET	/books	BookController@index	books.index	List all books
GET	/books/create	BookController@create	books.create	Show create book form
POST	/books	BookController@store	books.store	Store new book
GET	/books/{book}	BookController@show	books.show	Show book details
GET	/books/{book}/edit	BookController@edit	books.edit	Show edit book form
PUT	/books/{book}	BookController@update	books.update	Update book
DELETE	/books/{book}	BookController@destroy	books.destroy	Delete book
GET	/orders	OrderController@index	orders.index	List user orders
POST	/orders	OrderController@store	orders.store	Create new order
GET	/orders/{order}	OrderController@show	orders.show	Show order details
PATCH	/orders/{order}/status	OrderController@updateStatus	orders.updateStatus	Update order status
POST	/books/{book}/reviews	ReviewController@store	reviews.store	Store book review

HTTP Method	URI	Controller Method	Route Name	Description
DELETE	/reviews/{review}	ReviewController@destroy	reviews.destroy	Delete review

---

Part 1: Project Setup and Authentication (20 points)

Step 1.1: Create Laravel Project

Open your terminal and create a new Laravel project:

```
composer create-project laravel/laravel pageturner-bookstore
cd pageturner-bookstore
```

Step 1.2: Install Laravel Breeze

Install Laravel Breeze for authentication scaffolding:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
npm run build
```

Step 1.3: Configure Database

Edit your `.env` file to configure the database connection:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pageturner_bookstore
DB_USERNAME=root
DB_PASSWORD=your_password
```

Step 1.4: Modify Users Migration

Add a `role` column to the users table. Edit `database/migrations/xxxx_create_users_table.php`:

```
public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->enum('role', ['admin', 'customer'])-
>default('customer');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

Part 2: Database Migrations (20 points)

Step 2.1: Create Migrations

Generate migrations for all required tables:

```
php artisan make:migration create_categories_table
php artisan make:migration create_books_table
php artisan make:migration create_orders_table
php artisan make:migration create_order_items_table
php artisan make:migration create_reviews_table
```

Step 2.2: Define Migration Schemas

Categories Migration (`database/migrations/xxxx_create_categories_table.php`):

```
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description')->nullable();
        $table->timestamps();
    });
}
```

Books Migration (`database/migrations/xxxx_create_books_table.php`):

```
public function up(): void
{
    Schema::create('books', function (Blueprint $table) {
        $table->id();
        $table->foreignId('category_id')->constrained()-
>onDelete('cascade');
        $table->string('title');
        $table->string('author');
        $table->string('isbn')->unique();
        $table->decimal('price', 10, 2);
        $table->integer('stock_quantity')->default(0);
        $table->text('description')->nullable();
        $table->string('cover_image')->nullable();
        $table->timestamps();
    });
}
```

Orders Migration (`database/migrations/xxxx_create_orders_table.php`):

```
public function up(): void
{
    Schema::create('orders', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()-
>onDelete('cascade');
        $table->decimal('total_amount', 10, 2);
        $table->enum('status', ['pending', 'processing', 'completed',
'cancelled'])->default('pending');
        $table->timestamps();
    });
}
```

Order Items Migration (`database/migrations/xxxx_create_order_items_table.php`):

```
public function up(): void
{
    Schema::create('order_items', function (Blueprint $table) {
        $table->id();
        $table->foreignId('order_id')->constrained()-
>onDelete('cascade');
        $table->foreignId('book_id')->constrained()-
>onDelete('cascade');
        $table->integer('quantity');
        $table->decimal('unit_price', 10, 2);
        $table->timestamps();
    });
}
```

Reviews Migration (`database/migrations/xxxx_create_reviews_table.php`):

```
public function up(): void
{
    Schema::create('reviews', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()-
>onDelete('cascade');
        $table->foreignId('book_id')->constrained()-
>onDelete('cascade');
        $table->tinyInteger('rating')->unsigned(); // 1-5 stars
        $table->text('comment')->nullable();
        $table->timestamps();

        // Ensure one review per user per book
        $table->unique(['user_id', 'book_id']);
    });
}
```

Step 2.3: Run Migrations

```
php artisan migrate
```

Part 3: Models with Eloquent Relationships (20 points)

Step 3.1: Create Models

```
php artisan make:model Category
php artisan make:model Book
php artisan make:model Order
php artisan make:model OrderItem
php artisan make:model Review
```

Step 3.2: Define Model Relationships

User Model (`app/Models/User.php`) - Add relationships:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
        'role',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    // Relationships
    public function orders()
    {
        return $this->hasMany(Order::class);
    }

    public function reviews()
    {
        return $this->hasMany(Review::class);
    }
```

```
    // Helper method
    public function isAdmin()
    {
        return $this->role === 'admin';
    }
}
```

Category Model (`app/Models/Category.php`):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'description'];

    public function books()
    {
        return $this->hasMany(Book::class);
    }
}
```

Book Model (`app/Models/Book.php`):

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    use HasFactory;

    protected $fillable = [
        'category_id',
        'title',
        'author',
        'isbn',
        'price',
        'stock_quantity',
        'description',
        'cover_image',
    ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function reviews()
    {
        return $this->hasMany(Review::class);
    }

    public function orderItems()
    {
        return $this->hasMany(OrderItem::class);
    }

    // Accessor for average rating
    public function getAverageRatingAttribute()
    {
        return $this->reviews()->avg('rating') ?? 0;
```

```
    }
}
```

Order Model (`app/Models/Order.php`):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;

    protected $fillable = ['user_id', 'total_amount', 'status'];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function orderItems()
    {
        return $this->hasMany(OrderItem::class);
    }
}
```

OrderItem Model (`app/Models/OrderItem.php`):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class OrderItem extends Model
{
    use HasFactory;

    protected $fillable = ['order_id', 'book_id', 'quantity',
'unit_price'];

    public function order()
    {
        return $this->belongsTo(Order::class);
    }

    public function book()
    {
        return $this->belongsTo(Book::class);
    }

    // Accessor for subtotal
    public function getSubtotalAttribute()
    {
        return <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>t</mi><mi>h</mi><mi>i</mi><mi>s</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>q</mi><mi>u</mi><mi>a</
mi><mi>n</mi><mi>t</mi><mi>i</mi><mi>t</mi><mi>y</mi><mo>&#x0002A;</
mo></mrow></math></span>this->unit_price;
    }
}
```

Review Model (`app/Models/Review.php`):

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Review extends Model
{
    use HasFactory;

    protected $fillable = ['user_id', 'book_id', 'rating', 'comment'];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function book()
    {
        return $this->belongsTo(Book::class);
    }
}
```

Part 4: Factories and Seeders (15 points)

Step 4.1: Create Factories

```php
php artisan make:factory CategoryFactory
php artisan make:factory BookFactory
php artisan make:factory OrderFactory
php artisan make:factory ReviewFactory
```

CategoryFactory (`database/factories/CategoryFactory.php`):

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class CategoryFactory extends Factory
{
    public function definition(): array
    {
        $categories = ['Fiction', 'Non-Fiction', 'Science',
'Technology',
                       'Biography', 'History', 'Romance', 'Mystery',
                       'Self-Help', 'Children'];

        return [
            'name' => fake()->unique()->randomElement($categories),
            'description' => fake()->paragraph(),
        ];
    }
}
```

BookFactory (`database/factories/BookFactory.php`):

```php
<?php

namespace Database\Factories;

use App\Models\Category;
use Illuminate\Database\Eloquent\Factories\Factory;

class BookFactory extends Factory
{
    public function definition(): array
    {
        return [
            'category_id' => Category::factory(),
            'title' => fake()->sentence(3),
            'author' => fake()->name(),
            'isbn' => fake()->unique()->isbn13(),
            'price' => fake()->randomFloat(2, 9.99, 99.99),
            'stock_quantity' => fake()->numberBetween(0, 100),
            'description' => fake()->paragraphs(3, true),
            'cover_image' => null,
        ];
    }
}
```

ReviewFactory (`database/factories/ReviewFactory.php`):

```php
<?php

namespace Database\Factories;

use App\Models\Book;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class ReviewFactory extends Factory
{
    public function definition(): array
    {
        return [
            'user_id' => User::factory(),
            'book_id' => Book::factory(),
            'rating' => fake()->numberBetween(1, 5),
            'comment' => fake()->paragraph(),
        ];
    }
}
```

Step 4.2: Create Database Seeder

Edit `database/seeders/DatabaseSeeder.php`:

```
<?php

namespace Database\Seeders;

use App\Models\Book;
use App\Models\Category;
use App\Models\Review;
use App\Models\User;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // Create admin user
        User::factory()->create([
            'name' => 'Admin User',
            'email' => 'admin@pageturner.com',
            'role' => 'admin',
        ]);

        // Create customer users
        $customers = User::factory(10)->create(['role' => 'customer']);

        // Create categories
        $categories = Category::factory(8)->create();

        // Create books for each category
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</
mi><mi>o</mi><mi>r</mi><mi>i</mi><mi>e</mi><mi>s</mi><mo>&#x02212;</
mo><mo>&#x0003E;</mo><mi>e</mi><mi>a</mi><mi>c</mi><mi>h</mi><mo
stretchy="false">&#x00028;</mo><mi>f</mi><mi>u</mi><mi>n</mi><mi>c</
mi><mi>t</mi><mi>i</mi><mi>o</mi><mi>n</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>category) {
            Book::factory(5)->create(['category_id' => $category->id]);
        });

        // Create reviews
        $books = Book::all();
        <span class="math-inline" style="display: inline;"><math
```

```
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>u</mi><mi>s</mi><mi>t</mi><mi>o</
mi><mi>m</mi><mi>e</mi><mi>r</mi><mi>s</mi><mo>&#x02212;</
mo><mo>&#x0003E;</mo><mi>e</mi><mi>a</mi><mi>c</mi><mi>h</mi><mo
stretchy="false">&#x00028;</mo><mi>f</mi><mi>u</mi><mi>n</mi><mi>c</
mi><mi>t</mi><mi>i</mi><mi>o</mi><mi>n</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>customer) use
($books) {
            // Each customer reviews 3-5 random books
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</mi><mi>s</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>r</mi><mi>a</mi><mi>n</
mi><mi>d</mi><mi>o</mi><mi>m</mi><mo stretchy="false">&#x00028;</
mo><mi>r</mi><mi>a</mi><mi>n</mi><mi>d</mi><mo
stretchy="false">&#x00028;</mo><mn>3</mn><mo>&#x0002C;</mo><mn>5</
mn><mo stretchy="false">&#x00029;</mo><mo stretchy="false">&#x00029;</
mo><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>e</mi><mi>a</mi><mi>c</
mi><mi>h</mi><mo stretchy="false">&#x00028;</mo><mi>f</mi><mi>u</
mi><mi>n</mi><mi>c</mi><mi>t</mi><mi>i</mi><mi>o</mi><mi>n</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>book) use
($customer) {
                Review::factory()->create([
                    'user_id' => $customer->id,
                    'book_id' => $book->id,
                ]);
            });
        });
    }
}
```

Step 4.3: Run Seeder

```
php artisan db:seed
# or fresh migration with seeding
php artisan migrate:fresh --seed
```

Part 5: Controllers (25 points)

Step 5.1: Create Controllers

```
php artisan make:controller HomeController
php artisan make:controller CategoryController --resource
php artisan make:controller BookController --resource
php artisan make:controller OrderController --resource
php artisan make:controller ReviewController
```

Step 5.2: Implement Controllers

HomeController (`app/Http/Controllers/HomeController.php`):

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use App\Models\Category;

class HomeController extends Controller
{
    public function index()
    {
        $featuredBooks = Book::with('category')
            ->orderBy('created_at', 'desc')
            ->take(8)
            ->get();

        $categories = Category::withCount('books')->get();

        return view('home', compact('featuredBooks', 'categories'));
    }
}
```

CategoryController (`app/Http/Controllers/CategoryController.php`):

```
<?php

namespace App\Http\Controllers;

use App\Models\Category;
use Illuminate\Http\Request;

class CategoryController extends Controller
{
    public function index()
    {
        $categories = Category::withCount('books')->paginate(10);
        return view('categories.index', compact('categories'));
    }

    public function create()
    {
        return view('categories.create');
    }

    public function store(Request $request)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><mo>&#x0003D;</mo></mrow></
math></span>request->validate([
            'name' => 'required|string|max:255|unique:categories',
            'description' => 'nullable|string',
        ]);

        Category::create($validated);

        return redirect()->route('categories.index')
            ->with('success', 'Category created successfully!');
    }

    public function show(Category $category)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</
```

```
mi><mi>o</mi><mi>r</mi><mi>y</mi><mo>&#x02212;</mo><mo>&#x0003E;</
mo><mi>l</mi><mi>o</mi><mi>a</mi><mi>d</mi><mo
stretchy="false">&#x00028;</mo><msup><mo stretchy="false">[</
mo><mi>&#x02032;</mi></msup><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</
mi><msup><mi>s</mi><mi>&#x02032;</mi></msup><mo>&#x0003D;</
mo><mo>&#x0003E;</mo><mi>f</mi><mi>u</mi><mi>n</mi><mi>c</mi><mi>t</
mi><mi>i</mi><mi>o</mi><mi>n</mi><mo stretchy="false">&#x00028;</mo></
mrow></math></span>query) {
            $query->paginate(12);
        }]);

        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</mi><mi>s</
mi><mo>&#x0003D;</mo></mrow></math></span>category->books()-
>paginate(12);

        return view('categories.show', compact('category', 'books'));
    }

    public function edit(Category $category)
    {
        return view('categories.edit', compact('category'));
    }

    public function update(Request <span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>r</mi><mi>e</mi><mi>q</mi><mi>u</
mi><mi>e</mi><mi>s</mi><mi>t</mi><mo>&#x0002C;</mo><mi>C</mi><mi>a</
mi><mi>t</mi><mi>e</mi><mi>g</mi><mi>o</mi><mi>r</mi><mi>y</mi></
mrow></math></span>category)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline">><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><mo>&#x0003D;</mo></mrow></
math></span>request->validate([
            'name' => 'required|string|max:255|
unique:categories,name,' . $category->id,
            'description' => 'nullable|string',
        ]);
```

```
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</
mi><mi>o</mi><mi>r</mi><mi>y</mi><mo>&#x02212;</mo><mo>&#x0003E;</
mo><mi>u</mi><mi>p</mi><mi>d</mi><mi>a</mi><mi>t</mi><mi>e</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>validated);

        return redirect()->route('categories.index')
            ->with('success', 'Category updated successfully!');
    }

    public function destroy(Category $category)
    {
        $category->delete();

        return redirect()->route('categories.index')
            ->with('success', 'Category deleted successfully!');
    }
}
```

BookController (`app/Http/Controllers/BookController.php`):

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use App\Models\Category;
use Illuminate\Http\Request;

class BookController extends Controller
{
    public function index(Request $request)
    {
        $query = Book::with('category');

        // Filter by category if provided
        if ($request->has('category')) {
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>q</mi><mi>u</mi><mi>e</mi><mi>r</mi><mi>y</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>w</mi><mi>h</mi><mi>e</
mi><mi>r</mi><mi>e</mi><msup><mo stretchy="false">&#x00028;</
mo><mi>&#x02032;</mi></msup><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</
mi><mi>g</mi><mi>o</mi><mi>r</mi><msub><mi>y</mi><mi>i</mi></
msub><msup><mi>d</mi><mi>&#x02032;</mi></msup><mo>&#x0002C;</mo></
mrow></math></span>request->category);
        }

        // Search by title or author
        if ($request->has('search')) {
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>s</mi><mi>e</mi><mi>a</mi><mi>r</mi><mi>c</
mi><mi>h</mi><mo>&#x0003D;</mo></mrow></math></span>request->search;
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>q</mi><mi>u</mi><mi>e</mi><mi>r</mi><mi>y</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>w</mi><mi>h</mi><mi>e</
mi><mi>r</mi><mi>e</mi><mo stretchy="false">&#x00028;</mo><mi>f</
mi><mi>u</mi><mi>n</mi><mi>c</mi><mi>t</mi><mi>i</mi><mi>o</mi><mi>n</
mi><mo stretchy="false">&#x00028;</mo></mrow></math></span>q) use
($search) {
                <span class="math-inline" style="display:
```

```
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>q</mi><mo>&#x02212;</mo><mo>&#x0003E;</
mo><mi>w</mi><mi>h</mi><mi>e</mi><mi>r</mi><mi>e</mi><msup><mo
stretchy="false">&#x00028;</mo><mi>&#x02032;</mi></msup><mi>t</
mi><mi>i</mi><mi>t</mi><mi>l</mi><msup><mi>e</mi><mi>&#x02032;</mi></
msup><msup><mo>&#x0002C;</mo><mi>&#x02032;</mi></msup><mi>l</mi><mi>i</
mi><mi>k</mi><msup><mi>e</mi><mi>&#x02032;</mi></msup><mo>&#x0002C;</
mo><mi>"</mi></mrow></math></span>search}%")
                   ->orWhere('author', 'like', "%{$search}%");
            });
        }

        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</mi><mi>s</
mi><mo>&#x0003D;</mo></mrow></math></span>query->paginate(12);
        $categories = Category::all();

        return view('books.index', compact('books', 'categories'));
    }

    public function create()
    {
        $categories = Category::all();
        return view('books.create', compact('categories'));
    }

    public function store(Request $request)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><mo>&#x0003D;</mo></mrow></
math></span>request->validate([
            'category_id' => 'required|exists:categories,id',
            'title' => 'required|string|max:255',
            'author' => 'required|string|max:255',
            'isbn' => 'required|string|unique:books',
            'price' => 'required|numeric|min:0',
            'stock_quantity' => 'required|integer|min:0',
            'description' => 'nullable|string',
            'cover_image' => 'nullable|image|max:2048',
```

```
        ]);

        if ($request->hasFile('cover_image')) {
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><msup><mo
stretchy="false">[</mo><mi>&#x02032;</mi></msup><mi>c</mi><mi>o</
mi><mi>v</mi><mi>e</mi><msub><mi>r</mi><mi>i</mi></msub><mi>m</
mi><mi>a</mi><mi>g</mi><msup><mi>e</mi><mi>&#x02032;</mi></msup><mo
stretchy="false">]</mo><mo>&#x0003D;</mo></mrow></math></span>request-
>file('cover_image')
                ->store('covers', 'public');
        }

        Book::create($validated);

        return redirect()->route('books.index')
            ->with('success', 'Book added successfully!');
    }

    public function show(Book $book)
    {
        $book->load(['category', 'reviews.user']);
        return view('books.show', compact('book'));
    }

    public function edit(Book $book)
    {
        $categories = Category::all();
        return view('books.edit', compact('book', 'categories'));
    }

    public function update(Request <span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>r</mi><mi>e</mi><mi>q</mi><mi>u</
mi><mi>e</mi><mi>s</mi><mi>t</mi><mo>&#x0002C;</mo><mi>B</mi><mi>o</
mi><mi>o</mi><mi>k</mi></mrow></math></span>book)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
```

```
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><mo>&#x0003D;</mo></mrow></
math></span>request->validate([
            'category_id' => 'required|exists:categories,id',
            'title' => 'required|string|max:255',
            'author' => 'required|string|max:255',
            'isbn' => 'required|string|unique:books,isbn,' . $book->id,
            'price' => 'required|numeric|min:0',
            'stock_quantity' => 'required|integer|min:0',
            'description' => 'nullable|string',
            'cover_image' => 'nullable|image|max:2048',
        ]);

        if ($request->hasFile('cover_image')) {
            <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><msup><mo
stretchy="false">[</mo><mi>&#x02032;</mi></msup><mi>c</mi><mi>o</
mi><mi>v</mi><mi>e</mi><msub><mi>r</mi><mi>i</mi></msub><mi>m</
mi><mi>a</mi><mi>g</mi><msup><mi>e</mi><mi>&#x02032;</mi></msup><mo
stretchy="false">]</mo><mo>&#x0003D;</mo></mrow></math></span>request-
>file('cover_image')
                ->store('covers', 'public');
        }

        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>u</mi><mi>p</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mo stretchy="false">&#x00028;</mo></
mrow></math></span>validated);

        return redirect()->route('books.show', $book)
            ->with('success', 'Book updated successfully!');
    }

    public function destroy(Book $book)
    {
        $book->delete();

        return redirect()->route('books.index')
            ->with('success', 'Book deleted successfully!');
```

```
    }
}
```

ReviewController (`app/Http/Controllers/ReviewController.php`):

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use App\Models\Review;
use Illuminate\Http\Request;

class ReviewController extends Controller
{
    public function store(Request <span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>r</mi><mi>e</mi><mi>q</mi><mi>u</
mi><mi>e</mi><mi>s</mi><mi>t</mi><mo>&#x0002C;</mo><mi>B</mi><mi>o</
mi><mi>o</mi><mi>k</mi></mrow></math></span>book)
    {
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><mo>&#x0003D;</mo></mrow></
math></span>request->validate([
            'rating' => 'required|integer|min:1|max:5',
            'comment' => 'nullable|string|max:1000',
        ]);

        $validated['user_id'] = auth()->id();
        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>v</mi><mi>a</mi><mi>l</mi><mi>i</mi><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>d</mi><msup><mo
stretchy="false">[</mo><mi>&#x02032;</mi></msup><mi>b</mi><mi>o</
mi><mi>o</mi><msub><mi>k</mi><mi>i</mi></msub><msup><mi>d</
mi><mi>&#x02032;</mi></msup><mo stretchy="false">]</mo><mo>&#x0003D;</
mo></mrow></math></span>book->id;

        // Check if user already reviewed this book
        $existingReview = Review::where('user_id', auth()->id())
            ->where('book_id', $book->id)
            ->first();

        if ($existingReview) {
            <span class="math-inline" style="display: inline;"><math
```

```
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>e</mi><mi>x</mi><mi>i</mi><mi>s</mi><mi>t</
mi><mi>i</mi><mi>n</mi><mi>g</mi><mi>R</mi><mi>e</mi><mi>v</mi><mi>i</
mi><mi>e</mi><mi>w</mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>u</
mi><mi>p</mi><mi>d</mi><mi>a</mi><mi>t</mi><mi>e</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>validated);
            $message = 'Review updated successfully!';
        } else {
            Review::create($validated);
            $message = 'Review submitted successfully!';
        }

        return redirect()->route('books.show', $book)
            ->with('success', $message);
    }

    public function destroy(Review $review)
    {
        // Only allow owner or admin to delete
        if (auth()->id() !== $review->user_id && !auth()->user()-
>isAdmin()) {
            abort(403);
        }

        <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</
mi><mo>&#x0003D;</mo></mrow></math></span>review->book;
        $review->delete();

        return redirect()->route('books.show', $book)
            ->with('success', 'Review deleted successfully!');
    }
}
```

Part 6: Routes Configuration (10 points)

Edit `routes/web.php`:

```php
<?php

use App\Http\Controllers\BookController;
use App\Http\Controllers\CategoryController;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\OrderController;
use App\Http\Controllers\ProfileController;
use App\Http\Controllers\ReviewController;
use Illuminate\Support\Facades\Route;

// Public routes
Route::get('/', [HomeController::class, 'index'])->name('home');

// Book browsing (public)
Route::get('/books', [BookController::class, 'index'])-
>name('books.index');
Route::get('/books/{book}', [BookController::class, 'show'])-
>name('books.show');

// Category browsing (public)
Route::get('/categories', [CategoryController::class, 'index'])-
>name('categories.index');
Route::get('/categories/{category}', [CategoryController::class,
'show'])->name('categories.show');

// Authenticated routes
Route::middleware('auth')->group(function () {
    // Profile routes (from Breeze)
    Route::get('/profile', [ProfileController::class, 'edit'])-
>name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])-
>name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])-
>name('profile.destroy');

    // Review routes
    Route::post('/books/{book}/reviews', [ReviewController::class,
'store'])->name('reviews.store');
    Route::delete('/reviews/{review}', [ReviewController::class,
'destroy'])->name('reviews.destroy');

    // Order routes
```

```
    Route::get('/orders', [OrderController::class, 'index'])-
>name('orders.index');
    Route::post('/orders', [OrderController::class, 'store'])-
>name('orders.store');
    Route::get('/orders/{order}', [OrderController::class, 'show'])-
>name('orders.show');
});

// Admin-only routes (Category & Book management)
Route::middleware(['auth'])->prefix('admin')->name('admin.')-
>group(function () {
    // Category management
    Route::get('/categories/create', [CategoryController::class,
'create'])->name('categories.create');
    Route::post('/categories', [CategoryController::class, 'store'])-
>name('categories.store');
    Route::get('/categories/{category}/edit',
[CategoryController::class, 'edit'])->name('categories.edit');
    Route::put('/categories/{category}', [CategoryController::class,
'update'])->name('categories.update');
    Route::delete('/categories/{category}', [CategoryController::class,
'destroy'])->name('categories.destroy');

    // Book management
    Route::get('/books/create', [BookController::class, 'create'])-
>name('books.create');
    Route::post('/books', [BookController::class, 'store'])-
>name('books.store');
    Route::get('/books/{book}/edit', [BookController::class, 'edit'])-
>name('books.edit');
    Route::put('/books/{book}', [BookController::class, 'update'])-
>name('books.update');
    Route::delete('/books/{book}', [BookController::class, 'destroy'])-
>name('books.destroy');
});

require __DIR__.'/auth.php';
```

Part 7: Views and Blade Templates (40 points)

Step 7.1: Create Layout Template

Create `resources/views/layouts/app.blade.php`:

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <meta name="csrf-token" content="{{ csrf_token() }}">

    <title>@yield('title', 'PageTurner Bookstore')</title>

    <!-- Fonts -->
    <link rel="preconnect" href="https://fonts.bunny.net">
    <link href="https://fonts.bunny.net/css?family=figtree:
400,500,600&display=swap" rel="stylesheet" />

    <!-- Scripts -->
    @vite(['resources/css/app.css', 'resources/js/app.js'])

    @stack('styles')
</head>
<body class="font-sans antialiased bg-gray-100">
    <div class="min-h-screen">
        @include('partials.navigation')

        <!-- Page Heading -->
        @hasSection('header')
            <header class="bg-white shadow">
                <div class="max-w-7xl mx-auto py-6 px-4 sm:px-6
lg:px-8">
                    @yield('header')
                </div>
            </header>
        @endif

        <!-- Flash Messages -->
        @include('partials.flash-messages')

        <!-- Page Content -->
        <main class="py-6">
            <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
                @yield('content')
            </div>
```

```
        </main>

        @include('partials.footer')
    </div>

    @stack('scripts')
</body>
</html>
```

Step 7.2: Create Partial Views

Navigation Partial (`resources/views/partials/navigation.blade.php`):

```
<nav class="bg-indigo-600 text-white shadow-lg">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div class="flex justify-between h-16">
            <div class="flex items-center">
                <!-- Logo -->
                <a href="{{ route('home') }}" class="text-xl font-
bold">
                    PageTurner
                </a>

                <!-- Navigation Links -->
                <div class="hidden md:flex ml-10 space-x-4">
                    <a href="{{ route('home') }}" class="hover:bg-
indigo-700 px-3 py-2 rounded-md">
                        Home
                    </a>
                    <a href="{{ route('books.index') }}"
class="hover:bg-indigo-700 px-3 py-2 rounded-md">
                        Books
                    </a>
                    <a href="{{ route('categories.index') }}"
class="hover:bg-indigo-700 px-3 py-2 rounded-md">
                        Categories
                    </a>

                    @auth
                        @if(auth()->user()->isAdmin())
                            <a
href="{{ route('admin.books.create') }}" class="hover:bg-indigo-700
px-3 py-2 rounded-md">
                                Add Book
                            </a>
                            <a
href="{{ route('admin.categories.create') }}" class="hover:bg-
indigo-700 px-3 py-2 rounded-md">
                                Add Category
                            </a>
                        @endif
                    @endauth
                </div>
            </div>
```

```
            <!-- Right Side -->
            <div class="flex items-center space-x-4">
                @guest
                    <a href="{{ route('login') }}" class="hover:bg-
indigo-700 px-3 py-2 rounded-md">
                        Login
                    </a>
                    <a href="{{ route('register') }}" class="bg-white
text-indigo-600 px-4 py-2 rounded-md font-medium">
                        Register
                    </a>
                @endguest

                @auth
                    <a href="{{ route('orders.index') }}"
class="hover:bg-indigo-700 px-3 py-2 rounded-md">
                        My Orders
                    </a>
                    <span class="text-indigo-200">{{ auth()->user()-
>name }}</span>
                    <form method="POST" action="{{ route('logout') }}"
class="inline">
                        @csrf
                        <button type="submit" class="hover:bg-
indigo-700 px-3 py-2 rounded-md">
                            Logout
                        </button>
                    </form>
                @endauth
            </div>
        </div>
    </div>
</nav>
```

Flash Messages Partial (`resources/views/partials/flash-messages.blade.php`):

```
@if(session('success'))
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 mt-4">
        <div
class="bg-green-100 border border-green-400 text-green-700 px-4 py-3
rounded relative" role="alert">
            <span class="block sm:inline">{{ session('success') }}</
span>
        </div>
    </div>
@endif

@if(session('error'))
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 mt-4">
        <div class="bg-red-100 border border-red-400 text-red-700 px-4
py-3 rounded relative" role="alert">
            <span class="block sm:inline">{{ session('error') }}</span>
        </div>
    </div>
@endif

@if($errors->any())
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 mt-4">
        <div class="bg-red-100 border border-red-400 text-red-700 px-4
py-3 rounded relative">
            <ul class="list-disc list-inside">
                @foreach(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>e</mi><mi>r</mi><mi>r</mi><mi>o</mi><mi>r</
mi><mi>s</mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>a</mi><mi>l</
mi><mi>l</mi><mo stretchy="false">&#x00028;</mo><mo
stretchy="false">&#x00029;</mo><mi>a</mi><mi>s</mi></mrow></math></
span>error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    </div>
@endif
```

Footer Partial (`resources/views/partials/footer.blade.php`):

```
<footer class="bg-gray-800 text-white py-8 mt-12">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div class="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div>
                <h3 class="text-lg font-semibold mb-4">PageTurner
Bookstore</h3>
                <p class="text-gray-400">Your destination for quality
books at great prices.</p>
            </div>
            <div>
                <h3 class="text-lg font-semibold mb-4">Quick Links</h3>
                <ul class="space-y-2">
                    <li><a href="{{ route('home') }}" class="text-
gray-400 hover:text-white">Home</a></li>
                    <li><a href="{{ route('books.index') }}"
class="text-gray-400 hover:text-white">Browse Books</a></li>
                    <li><a href="{{ route('categories.index') }}"
class="text-gray-400 hover:text-white">Categories</a></li>
                </ul>
            </div>
            <div>
                <h3 class="text-lg font-semibold mb-4">Contact</h3>
                <p class="text-gray-400">Email:
support@pageturner.com</p>
                <p class="text-gray-400">Phone: (123) 456-7890</p>
            </div>
        </div>
        <div class="border-t border-gray-700 mt-8 pt-8 text-center
text-gray-400">
            <p>&copy; {{ date('Y') }} PageTurner Bookstore. All rights
reserved.</p>
        </div>
    </div>
</footer>
```

Step 7.3: Create Blade Components

Alert Component (`resources/views/components/alert.blade.php`):

```
@props(['type' => 'info'])

@php
    $classes = [
        'success' => 'bg-green-100 border-green-400 text-green-700',
        'error' => 'bg-red-100 border-red-400 text-red-700',
        'warning' => 'bg-yellow-100 border-yellow-400 text-yellow-700',
        'info' => 'bg-blue-100 border-blue-400 text-blue-700',
    ];
@endphp

<div {{ <span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>a</mi><mi>t</mi><mi>t</mi><mi>r</mi><mi>i</
mi><mi>b</mi><mi>u</mi><mi>t</mi><mi>e</mi><mi>s</mi><mo>&#x02212;</
mo><mo>&#x0003E;</mo><mi>m</mi><mi>e</mi><mi>r</mi><mi>g</mi><mi>e</
mi><mo stretchy="false">&#x00028;</mo><msup><mo stretchy="false">[</
mo><mi>&#x02032;</mi></msup><mi>c</mi><mi>l</mi><mi>a</mi><mi>s</
mi><msup><mi>s</mi><mi>&#x02032;</mi></msup><mo>&#x0003D;</
mo><msup><mo>&#x0003E;</mo><mi>&#x02032;</mi></msup><mi>b</mi><mi>o</
mi><mi>r</mi><mi>d</mi><mi>e</mi><mi>r</mi><mi>p</mi><mi>x</
mi><mo>&#x02212;</mo><mn>4</mn><mi>p</mi><mi>y</mi><mo>&#x02212;</
mo><mn>3</mn><mi>r</mi><mi>o</mi><mi>u</mi><mi>n</mi><mi>d</mi><mi>e</
mi><msup><mi>d</mi><mi>&#x02032;</mi></msup><mo>&#x0002E;</mo><mo
stretchy="false">&#x00028;</mo></mrow></math></span>classes[<span
class="math-inline" style="display: inline;"><math xmlns="http://
www.w3.org/1998/Math/MathML" display="inline"><mrow><mi>t</mi><mi>y</
mi><mi>p</mi><mi>e</mi><mo stretchy="false">]</mo><mo>&#x0003F;</
mo><mo>&#x0003F;</mo></mrow></math></span>classes['info'])]) }}>
    {{ $slot }}
</div>
```

Book Card Component (`resources/views/components/book-card.blade.php`):

```
@props(['book'])

<div class="bg-white rounded-lg shadow-md overflow-hidden hover:shadow-
lg transition-shadow">
    <div class="h-48 bg-gray-200 flex items-center justify-center">
        @if($book->cover_image)
            <img src="{{ asset('storage/' . <span class="math-inline"
style="font-family: serif;">book->cover_image) }}" alt="{{</span>book-
>title }}" class="h-full w-full object-cover">
        @else
            <svg class="h-20 w-20 text-gray-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168
5.477 3 6.253v13C4.168 18.477 5.754 18 7.5 18s3.332.477 4.5
1.253m0-13C13.168 5.477 14.754 5 16.5 5c1.747 0 3.332.477 4.5
1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746 0-3.332.477-4.5
1.253"></path>
            </svg>
        @endif
    </div>
    <div class="p-4">
        <h3 class="font-semibold text-lg text-gray-800 truncate">{{
$book->title }}</h3>
        <p class="text-gray-600 text-sm">by {{ $book->author }}</p>
        <p class="text-indigo-600 font-bold mt-2"><span class="math-
inline" style="display: inline;"><math xmlns="http://www.w3.org/1998/
Math/MathML" display="inline"><mrow><mrow><mrow><mi>n</mi><mi>u</
mi><mi>m</mi><mi>b</mi><mi>e</mi><msub><mi>r</mi><mi>f</mi></
msub><mi>o</mi><mi>r</mi><mi>m</mi><mi>a</mi><mi>t</mi><mo
stretchy="false">&#x00028;</mo></mrow></mrow></mrow></math></span>book-
>price, 2) }}</p>

        {{-- Star Rating --}}
        <div class="flex items-center mt-2">
            @for(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline">><mrow><mi>i</mi><mo>&#x0003D;</mo><mn>1</mn><mi>;</
mi></mrow></math></span>i <= 5; $i++)
                @if(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline">><mrow><mi>i</mi><mo>&#x0003C;</mo><mo>&#x0003D;</
```

```
mo><mi>r</mi><mi>o</mi><mi>u</mi><mi>n</mi><mi>d</mi><mo
stretchy="false">&#x00028;</mo></mrow></math></span>book-
>average_rating))
                    <svg class="h-4 w-4 text-yellow-400"
fill="currentColor" viewBox="0 0 20 20">
                        <path d="M9.049 2.927c.3-.921 1.603-.921 1.902
0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8
2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755 1.688-1.54
1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-.
197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.
38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z"/>
                    </svg>
                @else
                    <svg class="h-4 w-4 text-gray-300"
fill="currentColor" viewBox="0 0 20 20">
                        <path d="M9.049 2.927c.3-.921 1.603-.921 1.902
0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8
2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755 1.688-1.54
1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-.
197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.
38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z"/>
                    </svg>
                @endif
            @endfor
            <span class="ml-1 text-sm text-gray-500">({{ $book-
>reviews->count() }})</span>
        </div>

        <a href="{{ route('books.show', $book) }}" class="mt-4 block
text-center bg-indigo-600 text-white py-2 rounded hover:bg-indigo-700
transition">
            View Details
        </a>
    </div>
</div>
```

Step 7.4: Create Page Views

Home Page (`resources/views/home.blade.php`):

```
@extends('layouts.app')

@section('title', 'PageTurner - Online Bookstore')

@section('content')
    {{-- Hero Section --}}
    <div class="bg-indigo-700 text-white rounded-lg p-8 mb-8">
        <h1 class="text-4xl font-bold mb-4">Welcome to PageTurner</h1>
        <p class="text-xl text-indigo-200 mb-6">Discover your next
favorite book from our extensive collection.</p>
        <a href="{{ route('books.index') }}" class="bg-white text-
indigo-700 px-6 py-3 rounded-lg font-semibold hover:bg-indigo-100
transition">
            Browse Books
        </a>
    </div>

    {{-- Categories Section --}}
    <section class="mb-12">
        <h2 class="text-2xl font-bold mb-6">Browse by Category</h2>
        <div class="grid grid-cols-2 md:grid-cols-4 gap-4">
            @foreach(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</
mi><mi>o</mi><mi>r</mi><mi>i</mi><mi>e</mi><mi>s</mi><mi>a</mi><mi>s</
mi></mrow></math></span>category)
                <a href="{{ route('categories.show', $category) }}"
class="bg-white p-4 rounded-lg shadow hover:shadow-md transition text-
center">
                    <h3 class="font-semibold text-gray-800">{{
$category->name }}</h3>
                    <p class="text-sm text-gray-500">{{ $category-
>books_count }} books</p>
                </a>
            @endforeach
        </div>
    </section>

    {{-- Featured Books Section --}}
    <section>
        <h2 class="text-2xl font-bold mb-6">Featured Books</h2>
```

```
        @forelse(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>f</mi><mi>e</mi><mi>a</mi><mi>t</mi><mi>u</
mi><mi>r</mi><mi>e</mi><mi>d</mi><mi>B</mi><mi>o</mi><mi>o</mi><mi>k</
mi><mi>s</mi><mi>a</mi><mi>s</mi></mrow></math></span>book)
            @if($loop->first)
                <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-
cols-3 lg:grid-cols-4 gap-6">
            @endif

            <x-book-card :book="$book" />

            @if($loop->last)
                </div>
            @endif
        @empty
            <x-alert type="info">
                No books available at the moment. Check back soon!
            </x-alert>
        @endforelse
    </section>
@endsection
```

Books Index (`resources/views/books/index.blade.php`):

```
@extends('layouts.app')

@section('title', 'All Books - PageTurner')

@section('header')
    <h1 class="text-3xl font-bold text-gray-900">All Books</h1>
@endsection

@section('content')
    {{-- Search and Filter --}}
    <div class="bg-white p-4 rounded-lg shadow mb-6">
        <form action="{{ route('books.index') }}" method="GET"
class="flex flex-wrap gap-4">
            <div class="flex-1 min-w-[200px]">
                <input type="text" name="search"
value="{{ request('search') }}"
                       placeholder="Search by title or author..."
                       class="w-full border-gray-300 rounded-md shadow-
sm focus:ring-indigo-500 focus:border-indigo-500">
            </div>
            <div class="w-48">
                <select name="category" class="w-full border-gray-300
rounded-md shadow-sm focus:ring-indigo-500 focus:border-indigo-500">
                    <option value="">All Categories</option>
                    @foreach(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</
mi><mi>o</mi><mi>r</mi><mi>i</mi><mi>e</mi><mi>s</mi><mi>a</mi><mi>s</
mi></mrow></math></span>category)
                        <option value="{{ <span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</
mi><mi>g</mi><mi>o</mi><mi>r</mi><mi>y</mi><mo>&#x02212;</
mo><mo>&#x0003E;</mo><mi>i</mi><mi>d</mi><mi>}</mi><mi>}</mi><mi>"</
mi><mrow><mrow><mi>r</mi><mi>e</mi><mi>q</mi><mi>u</mi><mi>e</
mi><mi>s</mi><mi>t</mi><msup><mo stretchy="false">&#x00028;</
mo><mi>&#x02032;</mi></msup><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</
mi><mi>g</mi><mi>o</mi><mi>r</mi><msup><mi>y</mi><mi>&#x02032;</mi></
msup><mo stretchy="false">&#x00029;</mo><mo>&#x0003D;</
mo><mo>&#x0003D;</mo></mrow></mrow></mrow></math></span>category->id ?
'selected' : '' }}>
                            {{ $category->name }}
```

```
                        </option>
                    @endforeach
                </select>
            </div>
            <button type="submit" class="bg-indigo-600 text-white px-6
py-2 rounded-md hover:bg-indigo-700 transition">
                Search
            </button>
        </form>
    </div>

    {{-- Books Grid --}}
    @if($books->count() > 0)
        <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3
lg:grid-cols-4 gap-6">
            @foreach(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</mi><mi>s</
mi><mi>a</mi><mi>s</mi></mrow></math></span>book)
                <x-book-card :book="$book" />
            @endforeach
        </div>

        {{-- Pagination --}}
        <div class="mt-8">
            {{ $books->withQueryString()->links() }}
        </div>
    @else
        <x-alert type="info">
            No books found matching your criteria.
        </x-alert>
    @endif
@endsection
```

Book Detail Page (`resources/views/books/show.blade.php`):

```
@extends('layouts.app')

@section('title', $book->title . ' - PageTurner')

@section('content')
    <div class="bg-white rounded-lg shadow-lg overflow-hidden">
        <div class="md:flex">
            {{-- Book Cover --}}
            <div class="md:w-1/3 bg-gray-200 p-8 flex items-center
justify-center">
                @if($book->cover_image)
                    <img src="{{ asset('storage/' . <span class="math-
inline" style="font-family: serif;">book->cover_image) }}" alt="{{</
span>book->title }}" class="max-h-96 object-contain">
                @else
                    <svg class="h-48 w-48 text-gray-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-
linejoin="round" stroke-width="2" d="M12 6.253v13m0-13C10.832 5.477
9.246 5 7.5 5S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18 7.5
18s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5 16.5 5c1.747 0
3.332.477 4.5 1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746
0-3.332.477-4.5 1.253"></path>
                    </svg>
                @endif
            </div>

            {{-- Book Details --}}
            <div class="md:w-2/3 p-8">
                <span class="text-indigo-600 text-sm font-medium">{{
$book->category->name }}</span>
                <h1 class="text-3xl font-bold text-gray-900 mt-2">{{
$book->title }}</h1>
                <p class="text-xl text-gray-600 mt-1">by {{ $book-
>author }}</p>

                {{-- Rating --}}
                <div class="flex items-center mt-4">
                    @for(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>i</mi><mo>&#x0003D;</mo><mn>1</mn><mi>;</
mi></mrow></math></span>i <= 5; $i++)
```

```
                        <svg class="h-6 w-6 {{ <span class="math-
inline" style="display: inline;"><math xmlns="http://www.w3.org/1998/
Math/MathML" display="inline"><mrow><mi>i</mi><mo>&#x0003C;</
mo><mo>&#x0003D;</mo><mi>r</mi><mi>o</mi><mi>u</mi><mi>n</mi><mi>d</
mi><mo stretchy="false">&#x00028;</mo></mrow></math></span>book-
>average_rating) ? 'text-yellow-400' : 'text-gray-300' }}"
fill="currentColor" viewBox="0 0 20 20">
                            <path d="M9.049 2.927c.3-.921 1.603-.921
1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8
2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755 1.688-1.54
1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-.
197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.
38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z"/>
                        </svg>
                    @endfor
                    <span class="ml-2 text-gray-600">{{
number_format(<span class="math-inline" style="font-family:
serif;">book->average_rating, 1) }} ({{</span>book->reviews-
>count() }} reviews)</span>
                </div>

                <p class="text-3xl font-bold text-indigo-600
mt-4"><span class="math-inline" style="display: inline;"><math
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mrow><mrow><mi>n</mi><mi>u</mi><mi>m</
mi><mi>b</mi><mi>e</mi><msub><mi>r</mi><mi>f</mi></msub><mi>o</
mi><mi>r</mi><mi>m</mi><mi>a</mi><mi>t</mi><mo
stretchy="false">&#x00028;</mo></mrow></mrow></mrow></math></span>book-
>price, 2) }}</p>

                <div class="mt-4">
                    <span
class="text-sm {{ $book->stock_quantity > 0 ? 'text-green-600' : 'text-
red-600' }}">
                        @if($book->stock_quantity > 0)
                            In Stock ({{ $book->stock_quantity }}
available)
                        @else
                            Out of Stock
                        @endif
                    </span>
                </div>
```

```
                <div class="mt-4">
                    <p class="text-gray-600 text-sm"><strong>ISBN:</
strong> {{ $book->isbn }}</p>
                </div>

                <div class="mt-6">
                    <h3 class="font-semibold text-
gray-800">Description</h3>
                    <p class="text-gray-600 mt-2">{{ $book-
>description }}</p>
                </div>

                {{-- Admin Actions --}}
                @auth
                    @if(auth()->user()->isAdmin())
                        <div class="mt-6 flex space-x-4">
                            <a href="{{ route('admin.books.edit',
$book) }}" class="bg-yellow-500 text-white px-4 py-2 rounded hover:bg-
yellow-600 transition">
                                Edit Book
                            </a>
                            <form
action="{{ route('admin.books.destroy', $book) }}" method="POST"
onsubmit="return confirm('Are you sure?')">
                                @csrf
                                @method('DELETE')
                                <button type="submit"
class="bg-red-500 text-white px-4 py-2 rounded hover:bg-red-600
transition">
                                    Delete Book
                                </button>
                            </form>
                        </div>
                    @endif
                @endauth
            </div>
        </div>
    </div>

    {{-- Reviews Section --}}
    <div class="mt-8">
```

```
        <h2 class="text-2xl font-bold mb-6">Customer Reviews</h2>

        {{-- Review Form (for authenticated users) --}}
        @auth
            <div class="bg-white rounded-lg shadow p-6 mb-6">
                <h3 class="font-semibold text-lg mb-4">Write a Review</
h3>
                <form action="{{ route('reviews.store', $book) }}"
method="POST">
                    @csrf
                    <div class="mb-4">
                        <label class="block text-gray-700
mb-2">Rating</label>
                        <select name="rating" class="border-gray-300
rounded-md shadow-sm focus:ring-indigo-500 focus:border-indigo-500"
required>
                            <option value="">Select rating</option>
                            @for(<span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>i</mi><mo>&#x0003D;</mo><mn>5</
mn><mi>;</mi></mrow></math></span>i >= 1; $i--)
                                <option value="{{ <span class="math-
inline" style="font-family: serif;">i }}">{{</span>i }} Star{{ $i >
1 ? 's' : '' }}</option>
                            @endfor
                        </select>
                    </div>
                    <div class="mb-4">
                        <label class="block text-gray-700
mb-2">Comment</label>
                        <textarea name="comment" rows="4"
class="w-full border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500" placeholder="Share your thoughts
about this book..."></textarea>
                    </div>
                    <button type="submit" class="bg-indigo-600 text-
white px-6 py-2 rounded hover:bg-indigo-700 transition">
                        Submit Review
                    </button>
                </form>
            </div>
        @else
```

```
            <x-alert type="info" class="mb-6">
                <a href="{{ route('login') }}" class="text-indigo-600
hover:underline">Login</a> to write a review.
            </x-alert>
        @endauth

        {{-- Display Reviews --}}
        @forelse(<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</
mi><mo>&#x02212;</mo><mo>&#x0003E;</mo><mi>r</mi><mi>e</mi><mi>v</
mi><mi>i</mi><mi>e</mi><mi>w</mi><mi>s</mi><mi>a</mi><mi>s</mi></
mrow></math></span>review)
            <div class="bg-white rounded-lg shadow p-6 mb-4">
                <div class="flex justify-between items-start">
                    <div>
                        <p class="font-semibold">{{ $review->user-
>name }}</p>
                        <div class="flex items-center mt-1">
                            @for(<span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>i</mi><mo>&#x0003D;</mo><mn>1</
mn><mi>;</mi></mrow></math></span>i <= 5; $i++)
                                <svg class="h-4 w-4 {{ <span
class="math-inline" style="display: inline;"><math xmlns="http://
www.w3.org/1998/Math/MathML" display="inline"><mrow><mi>i</
mi><mo>&#x0003C;</mo><mo>&#x0003D;</mo></mrow></math></span>review-
>rating ? 'text-yellow-400' : 'text-gray-300' }}" fill="currentColor"
viewBox="0 0 20 20">
                                    <path d="M9.049 2.927c.3-.921
1.603-.921 1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371
1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755
1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.
784.57-1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98
8.72c-.783-.57-.38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z"/>
                                </svg>
                            @endfor
                        </div>
                    </div>
                    <div class="flex items-center space-x-2">
                        <span class="text-gray-500 text-sm">{{ $review-
>created_at->diffForHumans() }}</span>
```

```
                        @auth
                            @if(auth()->id() === $review->user_id ||
auth()->user()->isAdmin())
                                <form
action="{{ route('reviews.destroy', $review) }}" method="POST">
                                    @csrf
                                    @method('DELETE')
                                    <button type="submit" class="text-
red-500 hover:text-red-700 text-sm">Delete</button>
                                </form>
                            @endif
                        @endauth
                    </div>
                </div>
                @isset($review->comment)
                    <p class="text-gray-600 mt-3">{{ $review-
>comment }}</p>
                @endisset
            </div>
        @empty
            <x-alert type="info">
                No reviews yet. Be the first to review this book!
            </x-alert>
        @endforelse
    </div>
@endsection
```

Create/Edit Book Form (`resources/views/books/create.blade.php`):

```
@extends('layouts.app')

@section('title', 'Add New Book - PageTurner')

@section('header')
    <h1 class="text-3xl font-bold text-gray-900">Add New Book</h1>
@endsection

@section('content')
    <div class="max-w-2xl mx-auto">
        <div class="bg-white rounded-lg shadow p-6">
            <form action="{{ route('admin.books.store') }}"
method="POST" enctype="multipart/form-data">
                @csrf

                <div class="mb-4">
                    <label for="title" class="block text-gray-700 font-
medium mb-2">Title *</label>
                    <input type="text" name="title" id="title"
value="{{ old('title') }}"
                           class="w-full border-gray-300 rounded-md
shadow-sm focus:ring-indigo-500 focus:border-indigo-500 @error('title')
border-red-500 @enderror" required>
                    @error('title')
                        <p class="text-red-500 text-sm mt-1">{{
$message }}</p>
                    @enderror
                </div>

                <div class="mb-4">
                    <label for="author" class="block text-gray-700
font-medium mb-2">Author *</label>
                    <input type="text" name="author" id="author"
value="{{ old('author') }}"
                           class="w-full border-gray-300 rounded-md
shadow-sm focus:ring-indigo-500 focus:border-indigo-500" required>
                </div>

                <div class="mb-4">
                    <label for="category_id"
class="block text-gray-700 font-medium mb-2">Category *</label>
                    <select name="category_id" id="category_id"
```

```
                            class="w-full border-gray-300 rounded-md
shadow-sm focus:ring-indigo-500 focus:border-indigo-500" required>
                        <option value="">Select a category</option>
                        @foreach(<span class="math-inline"
style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/
MathML" display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</mi><mi>e</
mi><mi>g</mi><mi>o</mi><mi>r</mi><mi>i</mi><mi>e</mi><mi>s</mi><mi>a</
mi><mi>s</mi></mrow></math></span>category)
                            <option value="{{ <span class="math-
inline" style="display: inline;"><math xmlns="http://www.w3.org/1998/
Math/MathML" display="inline"><mrow><mi>c</mi><mi>a</mi><mi>t</
mi><mi>e</mi><mi>g</mi><mi>o</mi><mi>r</mi><mi>y</mi><mo>&#x02212;</
mo><mo>&#x0003E;</mo><mi>i</mi><mi>d</mi><mi>}</mi><mi>}</mi><mi>"</
mi><mrow><mrow><mi>o</mi><mi>l</mi><mi>d</mi><msup><mo
stretchy="false">&#x00028;</mo><mi>&#x02032;</mi></msup><mi>c</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mi>g</mi><mi>o</mi><mi>r</
mi><msub><mi>y</mi><mi>i</mi></msub><msup><mi>d</mi><mi>&#x02032;</
mi></msup><mo stretchy="false">&#x00029;</mo><mo>&#x0003D;</
mo><mo>&#x0003D;</mo></mrow></mrow></mrow></math></span>category->id ?
'selected' : '' }}>
                                {{ $category->name }}
                            </option>
                        @endforeach
                    </select>
                </div>

                <div class="grid grid-cols-2 gap-4 mb-4">
                    <div>
                        <label for="isbn" class="block text-gray-700
font-medium mb-2">ISBN *</label>
                        <input type="text" name="isbn" id="isbn"
value="{{ old('isbn') }}"

class="w-full border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500" required>
                    </div>
                    <div>
                        <label for="price" class="block text-gray-700
font-medium mb-2">Price ($) *</label>
                        <input type="number" step="0.01" name="price"
id="price" value="{{ old('price') }}"
```

```
class="w-full border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500" required>
                    </div>
                </div>

                <div class="mb-4">
                    <label for="stock_quantity" class="block text-
gray-700 font-medium mb-2">Stock Quantity *</label>
                    <input type="number" name="stock_quantity"
id="stock_quantity" value="{{ old('stock_quantity', 0) }}"
                           class="w-full border-gray-300 rounded-md
shadow-sm focus:ring-indigo-500 focus:border-indigo-500" required>
                </div>

                <div class="mb-4">
                    <label for="description"
class="block text-gray-700 font-medium mb-2">Description</label>
                    <textarea name="description" id="description"
rows="4"
                              class="w-full border-gray-300 rounded-md
shadow-sm focus:ring-indigo-500 focus:border-indigo-500">{{
old('description') }}</textarea>
                </div>

                <div class="mb-6">
                    <label for="cover_image"
class="block text-gray-700 font-medium mb-2">Cover Image</label>
                    <input type="file" name="cover_image"
id="cover_image" accept="image/*"
                           class="w-full border-gray-300 rounded-md
shadow-sm">
                </div>

                <div class="flex justify-end space-x-4">
                    <a href="{{ route('books.index') }}" class="bg-
gray-300 text-gray-700 px-6 py-2 rounded hover:bg-gray-400 transition">
                        Cancel
                    </a>
                    <button type="submit" class="bg-indigo-600 text-
white px-6 py-2 rounded hover:bg-indigo-700 transition">
                        Add Book
                    </button>
```

```
                </div>
            </form>
        </div>
    </div>
@endsection
```

Table 4: Blade Directives Used in This Activity

Directive	Purpose	Example Usage
`@extends`	Inherit layout template	`@extends('layouts.app')`
`@section`	Define content section	`@section('content')...@endsection`
`@yield`	Placeholder for section content	`@yield('content')`
`@include`	Include partial view	`@include('partials.navigation')`
`@if/@else/@endif`	Conditional rendering	`@if($book->stock > 0)...@endif`
`@foreach/@endforeach`	Loop through collections	`@foreach(<span class="math-inline" style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/MathML" display="inline"><mrow><mi>b</mi><mi>o</mi><mi>o</mi><mi>k</mi><mi>s</mi><mi>a</mi><mi>s</mi></mrow></math></span>book)...@endforeach`
`@forelse/@empty/@endforelse`	Loop with empty fallback	`@forelse(<span class="math-inline" style="display: inline;"><math xmlns="http://www.w3.org/1998/Math/MathML" display="inline"><mrow><mi>r</mi><mi>e</mi><mi>v</mi><mi>i</mi><mi>e</mi><mi>w</mi><mi>s</mi><mi>a</mi><mi>s</mi></mrow></math></span>review)...@empty...@endforelse`

Directive	Purpose	Example Usage
`@auth/ @endauth`	Content for authenticated users	`@auth...@endauth`
`@guest/ @endguest`	Content for guest users	`@guest...@endguest`
`@csrf`	CSRF token for forms	`@csrf`
`@method`	Spoof HTTP methods	`@method('DELETE')`
`@error/ @enderror`	Display validation errors	`@error('title')...@enderror`
`@isset/ @endisset`	Check if variable exists	`@isset($review->comment)...@endisset`
`@props`	Define component properties	`@props(['book'])`
`@stack/ @push`	Dynamic content injection	`@stack('scripts')` / `@push('scripts')`
`{{ }}`	Echo escaped content	`{{ $book->title }}`
`{!! !!}`	Echo unescaped content	`{!! $book->description !!}`

Table 5: Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)	Score
Project Setup & Authentication (20 pts)	Laravel Breeze installed correctly; user roles implemented; all configurations working	Minor issues with setup but functional	Setup complete with significant issues	Incomplete setup; authentication not working	/20
Database Migrations (20 pts)	All 5+ tables created with proper relationships, foreign keys, and constraints	Most tables correct with minor issues in relationships	Tables created but missing some foreign keys or constraints	Migrations fail or missing critical tables	/20
Eloquent Models & Relationships (20 pts)	All models defined with correct relationships (hasMany, belongsTo); accessors working	Most relationships correct; minor issues	Some relationships missing or incorrect	Models incomplete; relationships not working	/20
Factories & Seeders (15 pts)	All factories create realistic data; seeder populates database correctly with related data	Factories work with minor data issues	Basic seeding works but lacks realistic data	Seeders fail or don't populate related data	/15
Controllers (25 pts)	All CRUD operations implemented; proper validation; resource controllers used	Most operations work with minor issues	Basic CRUD works but missing validation	Controllers incomplete or not functioning	/25

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)	Score
Routes Configuration (10 pts)	All routes properly defined with names, grouping, and prefixes; authentication middleware applied	Most routes work; minor naming issues	Routes work but not properly organized	Missing routes or incorrect configurations	/10
Blade Views & Templates (40 pts)	Layout system with partials; components with slots; proper use of directives; responsive design	Good views with most features; minor styling issues	Basic views work but missing components or partials	Views incomplete or not using Blade features	/40
Code Quality & Standards (10 pts)	Clean, well-commented code; follows Laravel conventions; proper file organization	Good code quality with minor issues	Acceptable but inconsistent formatting	Poor code quality; hard to read/ maintain	/10
Functionality Testing (10 pts)	All features work correctly; proper error handling; user experience considered	Most features work with minor bugs	Basic functionality works	Many features broken or not implemented	/10
Documentation (5 pts)	Code comments where needed; README with setup instructions	Some documentation provided	Minimal documentation	No documentation	/5

Total Score: _____ / 175 points

Grading Scale

Grade	Percentage	Points Range
A	90-100%	158-175
B	80-89%	140-157
C	70-79%	123-139
D	60-69%	105-122
F	Below 60%	Below 105

Submission Requirements

Required Deliverables

1. **Complete Laravel Project** - Zipped folder containing:
   - All source code files
   - Database migrations
   - Models, Controllers, Views
   - Factories and Seeders
   - `.env.example` file (without actual credentials)
2. **Database Export** - SQL dump of the seeded database
3. **Screenshots** - Capture of:
   - Homepage with featured books
   - Book listing page
   - Book detail page with reviews
   - Category management (admin)
   - User authentication pages
4. **Video Demonstration** (Optional Bonus) - 3-5 minute walkthrough of the application

Submission Format

- Filename: `LastName_FirstName_Lab3_Bookstore.zip`
- Include a `README.md` with:
- Setup instructions
- Login credentials for test accounts
- Any additional notes

Additional Challenges (Bonus Points - Up to 25 pts)

1. **Shopping Cart Feature (+10 pts)**
   - Implement a session-based shopping cart
   - Allow users to add/remove books
   - Calculate totals
2. **Order Processing (+5 pts)**
   - Complete order workflow
   - Order status updates
   - Order history for users
3. **Advanced Search (+5 pts)**
   - Full-text search
   - Filter by price range
   - Sort options (price, rating, date)
4. **Image Upload with Storage (+5 pts)**
   - Proper file storage using Laravel Storage
   - Image resizing/thumbnails
   - Default placeholder images

References

- Laravel Documentation: https://laravel.com/docs
- Laravel Breeze: https://laravel.com/docs/starter-kits#laravel-breeze
- Blade Templates: https://laravel.com/docs/blade
- Eloquent ORM: https://laravel.com/docs/eloquent
- Database Migrations: https://laravel.com/docs/migrations

Prepared by: RMB
Date: February 2026
Version: 1.0